UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

Project Report

on

FoodBridge

Food Donation and Redistribution Website

Submitted by
ANMOL

25MCI10382

Under the Guidance Of
Mr. Sanjeev Kumar

Partial fulfilment for the award of the degree of
MASTER'S OF COMPUTER APPLICATION IN

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CHANDIGARH UNIVERSITY

October 2026

Certificate

This is to certify that Anmol, bearing UID 25MCI10382, of University Institute of Computing, Chandigarh University, has successfully completed the mini project entitled "FoodBridge – Food Donation and Redistribution Website" under the guidance of Mr. Sanjeev Kumar during the academic year 2025–2026, in partial fulfilment of the requirements for the degree of Master of Computer Application (MCA).

This project demonstrates the implementation of a full-stack web platform using HTML, CSS, JavaScript, Node.js and Express.js, which connects food donors with NGOs and needy people so that surplus food reaches those who need it. The student has shown dedication, creativity and a clear understanding of web development, REST API design and secure authentication throughout the project.

We hereby declare that this work is original and has been carried out by the student as part of the prescribed curriculum.

Project Guide:
Mr. Sanjeev Kumar
Assistant Professor

Acknowledgement

I would like to express my sincere gratitude to Mr. Sanjeev Kumar for his valuable guidance, continuous support and encouragement throughout the development of this project titled "FoodBridge." His suggestions and feedback helped me understand the concepts clearly and complete this project successfully.

I am also thankful to University Institute of Computing for providing the necessary resources, facilities and a positive learning environment that made this work possible.

This project helped me improve my understanding of full-stack web development, REST APIs, authentication and data handling. It was a great learning experience where I applied my knowledge to a real social problem.

I would also like to thank my friends and family for their constant support and motivation, and everyone who directly or indirectly contributed to the successful completion of this project.

Anmol
MCA [AI&ML]
UID: 25MCI10382
Chandigarh University

Table of Contents

Certificate 2
Acknowledgement 3
Contents 4
Abstract 5
Introduction 6
Tools and Technologies Used 7
Source Code 8
Result 11
Implementation Steps 14
GitHub Screenshot 15
Conclusion 16
References 17

Abstract

FoodBridge is a web-based platform designed to reduce food waste and help feed people in need. Every day, restaurants, hostels, caterers, wedding venues and households throw away edible food, while many people and NGOs struggle to arrange meals. This project connects the two sides on a single platform.

The system supports three types of users: Donors, NGOs and Needy people. Donors list surplus food with its quantity, pickup address and pickup deadline. NGOs and needy people search the available food by city, category or keyword and claim it. After the food is handed over, the donor marks the donation as delivered, and the home page updates a live counter of servings delivered.

The frontend is built with HTML, CSS and JavaScript. The backend is built with Node.js and Express.js and exposes a REST API. Passwords are hashed using bcrypt, and sessions are handled using JSON Web Tokens (JWT) with role-based access control. Donations that pass their pickup deadline are automatically marked as expired.

Overall, FoodBridge demonstrates how a simple, secure and interactive web application can create a practical social impact by reducing food waste and improving food access.

Introduction

Food waste and hunger exist side by side. A large quantity of edible food is discarded every day after events, in restaurants and in homes, while many people do not get enough meals. The main gap is not the lack of food but the lack of a fast and reliable way to connect surplus food with the people who need it before it spoils.

The FoodBridge project is developed to address this problem. It is a web application where a donor can list extra food in less than a minute, and an NGO or a needy person nearby can claim it immediately. Because every donation has a pickup deadline, the food is used while it is still fresh and safe.

Objectives of the project

- To reduce food waste by redistributing surplus food.
- To provide a simple platform for donors, NGOs and needy people.
- To provide secure registration and login with role-based access.
- To allow searching and filtering of available food by city, type and keyword.
- To show the impact through live statistics of servings delivered.

Scope: The current version supports donor, NGO and receiver accounts, donation listing, claiming, delivery confirmation and automatic expiry. It can be extended with maps, image upload, email or SMS alerts and an admin panel.

Tools and Technologies Used

Programming Language: JavaScript – used for both the frontend and the backend of the application.

Frontend (User Interface): HTML5 and CSS3 are used to build a responsive single-page interface. Vanilla JavaScript with the Fetch API communicates with the backend, updates the page without reloading and shows notifications.

Backend: Node.js is the runtime that executes the server code. Express.js is the framework used to create REST API routes and to serve the frontend files.

Security: bcryptjs hashes passwords before saving them, so the real password is never stored. jsonwebtoken (JWT) creates a signed login token that is checked on every protected request.

Database: A JSON file (data/db.json) stores users and donations. It needs no installation and can be replaced by MongoDB or MySQL for production use.

Development Environment: Visual Studio Code, with the terminal used to run npm commands.

Operating System and Browser: Windows; Google Chrome for testing.

API Routes

Method	Route	Purpose	Access
POST	/api/register	Create account	Public
POST	/api/login	Login and receive token	Public
GET	/api/donations	List and search available food	Public
POST	/api/donations	Add a donation	Donor
POST	/api/donations/:id/claim	Claim a donation	NGO, Needy
POST	/api/donations/:id/deliver	Mark as delivered	Donor
DELETE	/api/donations/:id	Delete available donation	Donor
GET	/api/my	Own donations or claims	Logged in
GET	/api/stats	Live statistics	Public

Source Code

Project structure: foodbridge/server.js, foodbridge/package.json, foodbridge/public/index.html

package.json

```
{
  "name": "foodbridge", "version": "1.0.0", "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "bcryptjs": "^2.4.3", "express": "^4.19.2", "jsonwebtoken": "^9.0.2" }
}
```

server.js (Backend)

```
const express = require('express'), fs = require('fs'), path = require('path');
const bcrypt = require('bcryptjs'), jwt = require('jsonwebtoken');
const app = express(), SECRET = process.env.JWT_SECRET || 'change-this-secret';
const FILE = path.join(__dirname, 'data', 'db.json');
// Simple JSON database (can be swapped with MongoDB/MySQL)
fs.mkdirSync(path.dirname(FILE), { recursive: true });
let db = fs.existsSync(FILE) ? JSON.parse(fs.readFileSync(FILE)) : { users: [], donations: [] };
const save = () => fs.writeFileSync(FILE, JSON.stringify(db, null, 2));
const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 6);
const clean = u => ({ id: u.id, name: u.name, email: u.email, role: u.role, city: u.city, phone: u.phone });
const sign = u => jwt.sign(clean(u), SECRET, { expiresIn: '7d' });
const view = d => ({ ...d, status: d.status === 'available' && new Date(d.expiresAt) < new Date()
  ? 'expired' : d.status });
const err = (res, code, error) => res.status(code).json({ error });
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// ---- JWT auth middleware with optional role check ----
const auth = roles => (req, res, next) => {
  try {
    const u = jwt.verify((req.headers.authorization || '').split(' ')[1], SECRET);
    if (roles && !roles.includes(u.role)) return err(res, 403, 'Your account type cannot do this.');
    req.user = u; next();
  } catch { err(res, 401, 'Please log in first.'); }
};

// ---- Register and login ----
app.post('/api/register', async (req, res) => {
  const { name, email, password, role, city, phone } = req.body;
  if (!name || !email || !password || !city) return err(res, 400, 'Name, email, password and city required.');
  if (!['donor', 'ngo', 'needy'].includes(role)) return err(res, 400, 'Choose a valid account type.');
  if (password.length < 6) return err(res, 400, 'Password must be at least 6 characters.');
  if (db.users.some(u => u.email === email.toLowerCase())) return err(res, 409, 'Email is already registered.');
  const u = { id: uid(), name, email: email.toLowerCase(), password: await bcrypt.hash(password, 10),
    role, city, phone: phone || '' };
  db.users.push(u); save(); res.json({ token: sign(u), user: clean(u) });
});
app.post('/api/login', async (req, res) => {
  const u = db.users.find(x => x.email === (req.body.email || '').toLowerCase());
  if (!u || !(await bcrypt.compare(req.body.password || '', u.password)))
    return err(res, 401, 'Wrong email or password.');
  res.json({ token: sign(u), user: clean(u) });
});

// ---- Donations ----
app.get('/api/donations', (req, res) => {
  const { q = '', city = '', category = '' } = req.query, has = (s, t) => s.toLowerCase().includes(t.toLowerCase());
  res.json(db.donations.map(view).filter(d => d.status === 'available' && (!category || d.category === category)
    && has(d.city, city) && has(d.title + d.description + d.address, q))
    .sort((a, b) => new Date(a.expiresAt) - new Date(b.expiresAt)));
});
app.get('/api/my', auth(), (req, res) => {
  const key = req.user.role === 'donor' ? 'donorId' : 'claimedBy';
  res.json(db.donations.filter(d => d[key] === req.user.id).map(view).reverse());
});
app.post('/api/donations', auth(['donor']), (req, res) => {
  const { title, category, quantity, address, city, expiresAt, description } = req.body;
  if (!title || !quantity || !address || !city || !expiresAt) return err(res, 400, 'Fill in all required fields.');
  if (new Date(expiresAt) < new Date()) return err(res, 400, 'Pickup deadline must be in the future.');
  const d = { id: uid(), title, category: category || 'cooked', quantity: Number(quantity), address, city,
    expiresAt, description: description || '', donorId: req.user.id, donorName: req.user.name,
    donorPhone: req.user.phone || '', status: 'available', claimedBy: null };
  db.donations.push(d); save(); res.json(d);
});
app.post('/api/donations/:id/claim', auth(['ngo', 'needy']), (req, res) => {
  const d = db.donations.find(x => x.id === req.params.id);
  if (!d) return err(res, 404, 'Donation not found.');
  if (view(d).status !== 'available') return err(res, 409, 'This food is no longer available.');
  Object.assign(d, { status: 'claimed', claimedBy: req.user.id, claimedByName: req.user.name,
    claimedPhone: req.user.phone || '' });
  save(); res.json(d);
});
app.post('/api/donations/:id/deliver', auth(['donor']), (req, res) => {
  const d = db.donations.find(x => x.id === req.params.id && x.donorId === req.user.id);
```

```
  if (!d || d.status !== 'claimed') return err(res, 400, 'Only claimed donations can be marked delivered.');
  d.status = 'delivered'; save(); res.json(d);
});
app.delete('/api/donations/:id', auth(['donor']), (req, res) => {
  const i = db.donations.findIndex(x => x.id === req.params.id && x.donorId === req.user.id
    && x.status === 'available');
  if (i < 0) return err(res, 400, 'Only available donations can be deleted.');
  db.donations.splice(i, 1); save(); res.json({ ok: true });
});
app.get('/api/stats', (req, res) => res.json({
  meals: db.donations.filter(d => d.status === 'delivered').reduce((s, d) => s + d.quantity, 0),
  donations: db.donations.length,
  donors: db.users.filter(u => u.role === 'donor').length,
  receivers: db.users.filter(u => u.role !== 'donor').length,
}));
app.listen(3000, () => console.log('FoodBridge running on http://localhost:3000'));
```

public/index.html (Frontend)

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>FoodBridge - Share surplus food with people who need it</title>
<style>
:root{--ink:#14231c;--muted:#5d6f66;--green:#1b6b4a;--dark:#0f4a32;--gold:#f2a93b;--line:#dde6df;--red:#c43d3d}
*{box-sizing:border-box;margin:0}body{font:16px/1.5 system-ui,sans-serif;background:#f4f7f4;color:var(--ink)}
nav{position:sticky;top:0;background:var(--dark);color:#fff;display:flex;justify-content:space-between;
padding:.8rem 5vw}nav div{display:flex;gap:.6rem;align-items:center}
.logo{font-weight:800;font-size:1.3rem;cursor:pointer}.logo span{color:var(--gold)}
.btn{border:0;border-radius:8px;padding:.55rem 1rem;font:inherit;font-weight:600;cursor:pointer;
background:var(--green);color:#fff}.btn.alt{background:var(--gold);color:var(--ink)}
.btn.danger{background:var(--red)}.btn.ghost{background:none;border:1.5px solid}
.hero{background:var(--dark);color:#fff;padding:4rem 5vw;display:grid;grid-template-columns:1.2fr 1fr;gap:3rem}
.counter{background:#fff;color:var(--ink);border-radius:18px;padding:1.8rem;text-align:center;
border-bottom:8px solid var(--gold)}.counter big{display:block;font-size:4rem;font-weight:800;color:var(--green)}
main{padding:2rem 5vw;max-width:1200px;margin:auto}label{font-size:.9rem;font-weight:600}
input,select,textarea{font:inherit;padding:.6rem;border:1.5px solid var(--line);border-radius:8px;width:100%}
.filters{display:flex;gap:.7rem;margin:1rem 0}.full{grid-column:1/-1}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(290px,1fr));gap:1.2rem}
.form{display:grid;grid-template-columns:1fr 1fr;gap:.8rem}.meta{color:var(--muted);font-size:.92rem}
.card,.panel{background:#fff;border:1px solid var(--line);border-radius:12px;padding:1.2rem}
.card{display:flex;flex-direction:column;gap:.5rem}
.tag{padding:.1rem .6rem;border-radius:99px;font-size:.8rem;background:#e3f2e9;width:fit-content}
dialog{border:0;border-radius:14px;padding:1.6rem;width:min(420px,92vw);margin:auto}
dialog form,#reg-fields{display:grid;gap:.7rem}.hide{display:none!important}
#toast{position:fixed;bottom:1rem;left:50%;transform:translateX(-50%);color:#fff;padding:.8rem 1.3rem;
border-radius:10px;display:none}@media(max-width:800px){.hero,.form{grid-template-columns:1fr}}
</style></head>
<body>
<nav><div class="logo" onclick="show('home')">Food<span>Bridge</span></div><div id="navbar"></div></nav>
<section id="home">
  <header class="hero"><div><h1>Surplus food should reach a plate, not a bin.</h1>
    <p>Donors list extra food. NGOs and families nearby claim it before it expires.</p><br>
    <button class="btn alt" onclick="openAuth('register')">Join FoodBridge</button></div>
  <div class="counter"><small>Servings delivered so far</small><big id="s-meals">0</big>
    <small><b id="s-donors">0</b> donors | <b id="s-recv">0</b> receivers | <b id="s-don">0</b> listings</small>
  </div></header>
  <main><h2>Food available now</h2>
    <div class="filters"><input id="f-q" placeholder="Search food, e.g. rice, bread">
      <input id="f-city" placeholder="City"><select id="f-cat"><option value="">All types</option>
      <option value="cooked">Cooked meals</option><option value="bakery">Bakery</option>
      <option value="produce">Fruits and vegetables</option><option value="packaged">Packaged food</option>
    </select></div><div id="list" class="grid"></div></main>
</section>
<section id="dash" class="hide"><main id="dash-body"></main></section>
<dialog id="auth"><form id="auth-form">
  <div style="display:flex;gap:.5rem"><button type="button" class="btn" onclick="authTab('login')">Log in</button>
  <button type="button" class="btn" onclick="authTab('register')">Sign up</button></div>
  <div id="reg-fields" class="hide"><input name="name" placeholder="Full name or organisation">
    <select name="role"><option value="donor">I am a donor</option><option value="ngo">I represent an NGO</option>
    <option value="needy">I need food for myself or family</option></select>
    <input name="city" placeholder="City"><input name="phone" placeholder="Phone number"></div>
  <input name="email" type="email" placeholder="Email" required>
  <input name="password" type="password" placeholder="Password (min 6 characters)" required>
  <button class="btn" id="auth-submit">Log in</button>
  <button type="button" class="btn ghost" onclick="auth.close()">Cancel</button></form></dialog>
<div id="toast"></div>
<script>
const $ = s => document.querySelector(s);
let token = localStorage.fb_t, user = JSON.parse(localStorage.fb_u || 'null'), mode = 'login';
const ICON = { cooked: '\u{1F35B}', bakery: '\u{1F35E}', produce: '\u{1F966}', packaged: '\u{1F4E6}' };
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => '&#' + c.charCodeAt(0) + ';');
const fmt = d => new Date(d).toLocaleString([], { dateStyle: 'medium', timeStyle: 'short' });
const fld = (l, h, c = '') => `<div class="${c}"><label>${l}</label>${h}</div>`;
function toast(m, bad) {
  const t = $('#toast'); t.textContent = m; t.style.display = 'block';
  t.style.background = bad ? '#c43d3d' : '#14231c'; clearTimeout(t.h);
  t.h = setTimeout(() => t.style.display = 'none', 3000);
}
async function api(path, method = 'GET', body) {
  const r = await fetch('/api' + path, { method, body: body && JSON.stringify(body),
```

```
    headers: { 'Content-Type': 'application/json', ...(token && { Authorization: 'Bearer ' + token }) } });
  const d = await r.json();
  if (!r.ok) { if (r.status === 401 && token) logout(true); throw new Error(d.error || 'Something went wrong'); }
  return d;
}
function show(v) {
  $('#home').classList.toggle('hide', v !== 'home'); $('#dash').classList.toggle('hide', v !== 'dash');
  window.scrollTo(0, 0); v === 'dash' ? loadDash() : (loadStats(), loadList());
}
function nav() {
  $('#navbar').innerHTML = user
    ? `<span>Hi, ${esc(user.name.split(' ')[0])}</span><button class="btn alt" onclick="show('dash')">My dashboard
       </button><button class="btn ghost" onclick="logout()">Log out</button>`
    : `<button class="btn ghost" onclick="openAuth('login')">Log in</button>
       <button class="btn alt" onclick="openAuth('register')">Sign up</button>`;
}
// ---- Auth ----
function openAuth(m) { authTab(m); $('#auth').showModal(); }
function authTab(m) {
  mode = m; $('#reg-fields').classList.toggle('hide', m !== 'register');
  $('#auth-submit').textContent = m === 'login' ? 'Log in' : 'Create account';
}
$('#auth-form').onsubmit = async e => {
  e.preventDefault();
  try {
    const d = await api('/' + mode, 'POST', Object.fromEntries(new FormData(e.target)));
    token = d.token; user = d.user; localStorage.fb_t = token; localStorage.fb_u = JSON.stringify(user);
    $('#auth').close(); e.target.reset(); nav(); toast('Welcome, ' + user.name + '!'); show('dash');
  } catch (er) { toast(er.message, true); }
};
function logout(exp) {
  token = user = null; localStorage.clear(); nav(); show('home'); toast(exp ? 'Session expired.' : 'Logged out.');
}
// ---- Cards, actions and listing ----
function card(d, ctx) {
  let act = '', b = (t, f, c = '') => `<button class="btn ${c}" onclick="${f}">${t}</button>`;
  if (ctx === 'public') act = !user ? b('Log in to claim', "openAuth('login')")
    : user.role !== 'donor' ? b('Claim this food', `action('${d.id}','claim')`) : '';
  else if (user.role === 'donor') act = d.status === 'available' ? b('Delete', `del('${d.id}')`, 'danger')
    : d.status === 'claimed' ? b('Mark delivered', `action('${d.id}','deliver')`) : '';
  const who = ctx === 'dash' && d.status !== 'available' ? `<div class="meta">${user.role === 'donor'
    ? 'Claimed by ' + esc(d.claimedByName) + ' ' + esc(d.claimedPhone) : 'Call: ' + esc(d.donorPhone)}</div>` : '';
  return `<article class="card"><span class="tag">${d.status}</span><h3>${ICON[d.category]} ${esc(d.title)}</h3>
    <div class="meta">${d.quantity} servings | ${esc(d.donorName)}</div>
    <div class="meta">\u{1F4CD} ${esc(d.address)}, ${esc(d.city)}</div>
    <div class="meta">\u{23F0} Pick up before ${fmt(d.expiresAt)}</div>
    ${d.description ? `<p>${esc(d.description)}</p>` : ''}${who}<div>${act}</div></article>`;
}
async function action(id, what) {
  try {
    await api(`/donations/${id}/${what}`, 'POST'); toast('Done: ' + what);
    what === 'claim' ? loadList() : loadDash();
  } catch (e) { toast(e.message, true); }
}
async function del(id) {
  if (!confirm('Delete this donation?')) return;
  try { await api('/donations/' + id, 'DELETE'); toast('Deleted.'); loadDash(); }
  catch (e) { toast(e.message, true); }
}
async function loadStats() {
  const s = await api('/stats');
  $('#s-meals').textContent = s.meals; $('#s-donors').textContent = s.donors;
  $('#s-recv').textContent = s.receivers; $('#s-don').textContent = s.donations;
}
async function loadList() {
  const p = new URLSearchParams({ q: $('#f-q').value, city: $('#f-city').value, category: $('#f-cat').value });
  const list = await api('/donations?' + p);
  $('#list').innerHTML = list.map(d => card(d, 'public')).join('') || '<p class="meta">No food listed.</p>';
}
let timer; ['#f-q', '#f-city'].forEach(s => $(s).oninput = () => {
  clearTimeout(timer); timer = setTimeout(loadList, 300); });
$('#f-cat').onchange = loadList;
// ---- Dashboard ----
async function loadDash() {
  if (!user) return openAuth('login');
  const mine = await api('/my'), donor = user.role === 'donor';
  $('#dash-body').innerHTML = `<h2>${donor ? 'Your donations' : 'Food you claimed'}</h2>
  <p class="meta">${esc(user.name)} | ${esc(user.city)} | ${user.role}</p>
  ${donor ? `<div class="panel"><h3>Add surplus food</h3><form class="form" id="don-form">
    ${fld('What food is it?', '<input name="title" required>', 'full')}
    ${fld('Type', `<select name="category"><option value="cooked">Cooked meals</option><option value="bakery">Bakery
      </option><option value="produce">Fruits and vegetables</option><option value="packaged">Packaged food</option>
      </select>`)}${fld('Servings', '<input name="quantity" type="number" min="1" required>')}
    ${fld('Pickup address', '<input name="address" required>')}
    ${fld('City', `<input name="city" value="${esc(user.city)}" required>`)}
    ${fld('Pick up before', '<input name="expiresAt" type="datetime-local" required>', 'full')}
    ${fld('Notes (optional)', '<textarea name="description" rows="2"></textarea>', 'full')}
    <div class="full"><button class="btn">Publish donation</button></div></form></div>` : ''}
  <div class="grid">${mine.map(d => card(d, 'dash')).join('') || '<p class="meta">Nothing yet.</p>'}</div>`;
  const f = $('#don-form');
  if (f) f.onsubmit = async e => {
```

```
    e.preventDefault();
    try { await api('/donations', 'POST', Object.fromEntries(new FormData(f))); toast('Published.'); loadDash(); }
    catch (er) { toast(er.message, true); }
  };
}
nav(); loadStats(); loadList();
</script></body></html>
```

Result

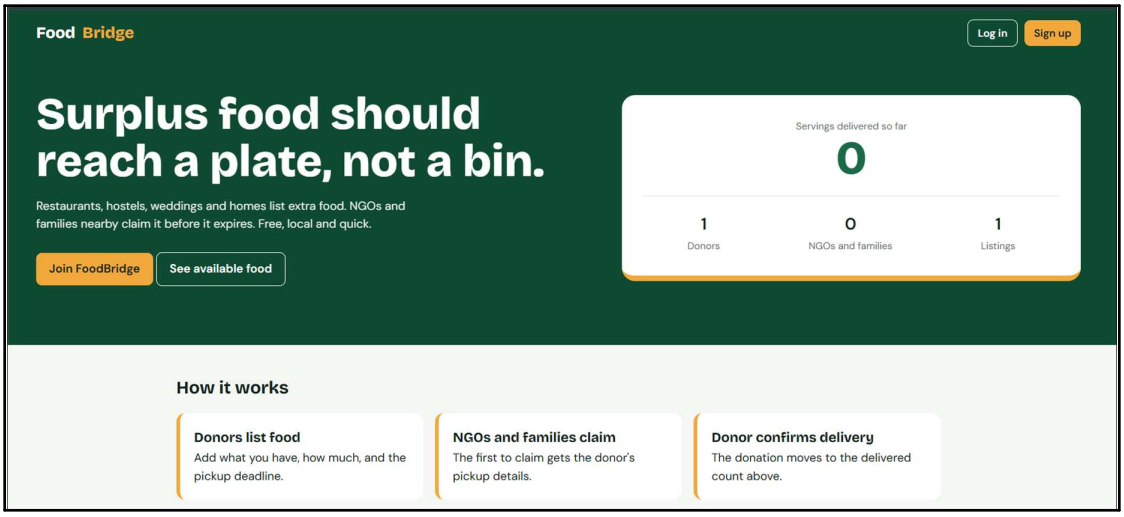


Fig. 1: Home page

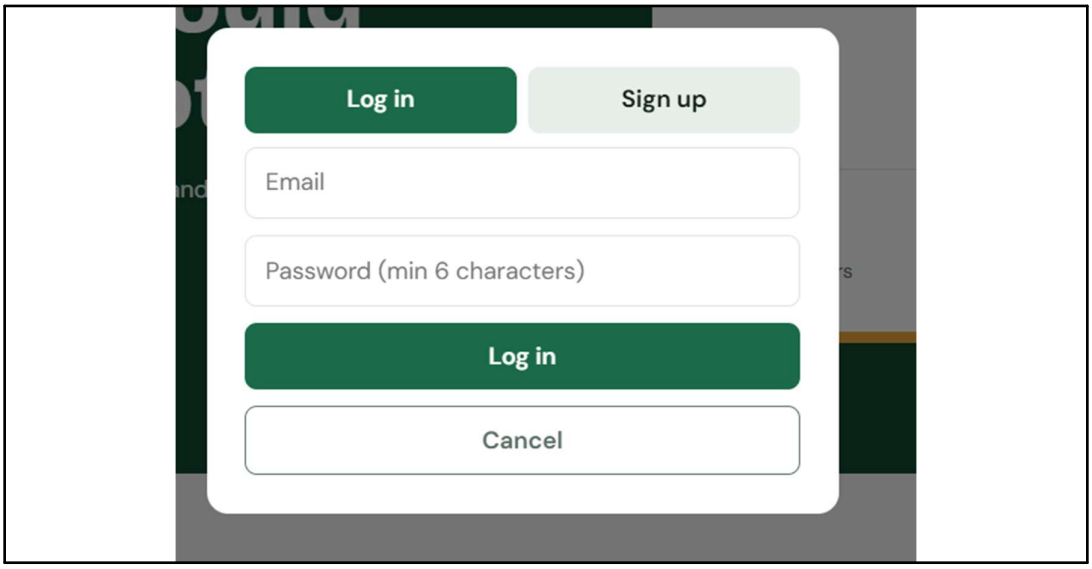


Fig. 2: Registration and login

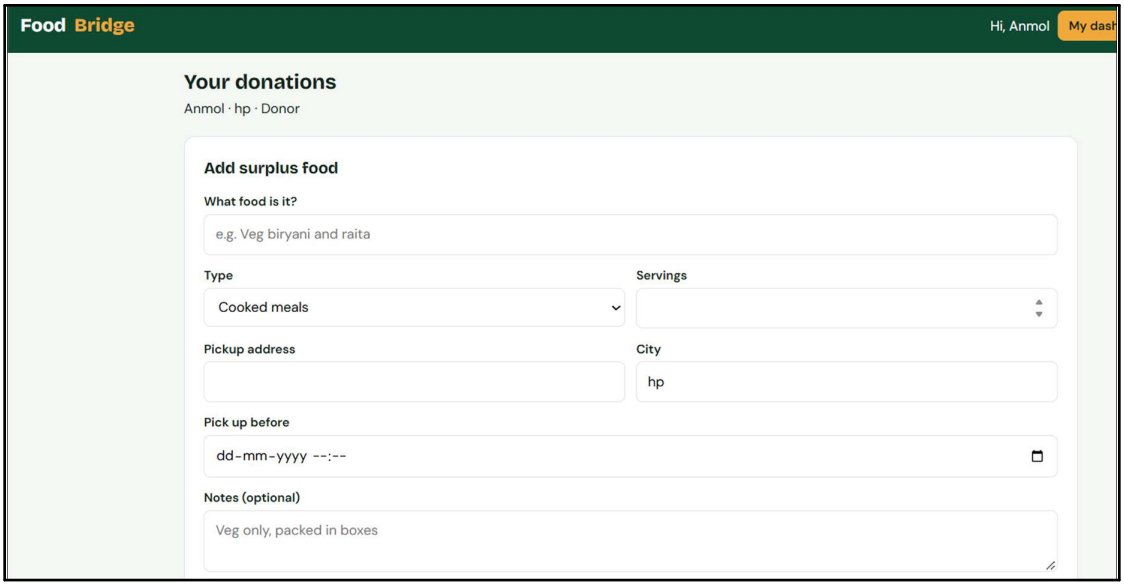


Fig. 3: Donor dashboard

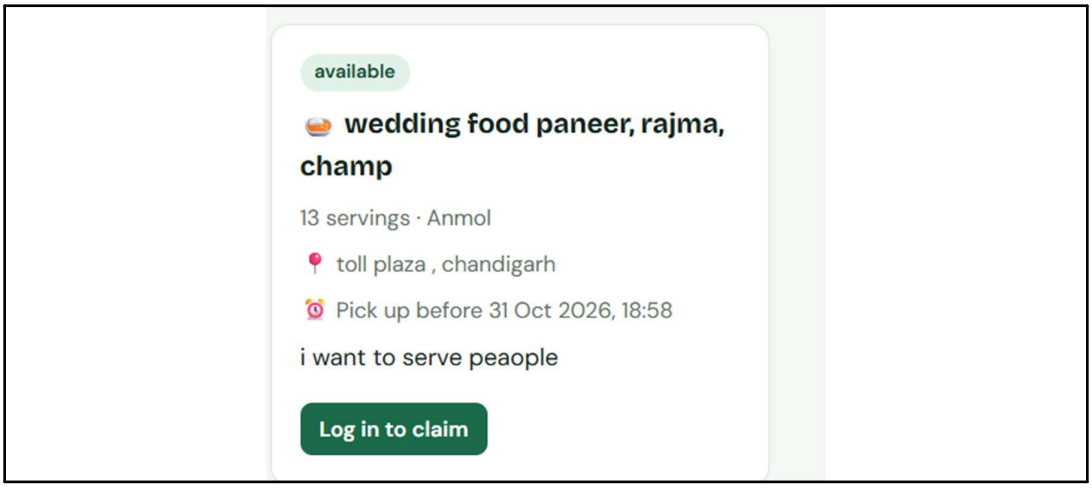


Fig. 4: Claiming a donation

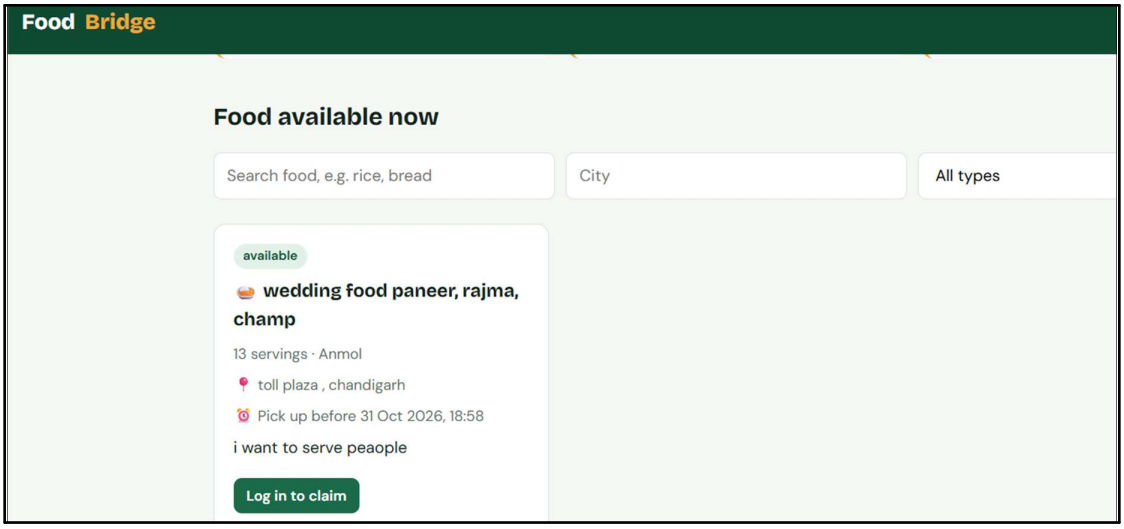


Fig. 5: Delivered status and live counter

Fig. 1 shows the home page of FoodBridge. The top section explains the purpose of the platform and displays a live counter of servings delivered, along with the number of donors, receivers and listings. Below it, the available food is shown as cards with the food name, servings, pickup address and pickup deadline. The user can search by keyword, city and food type.

Fig. 2 shows the registration and login window. A new user selects an account type: Donor, NGO or Needy person. Passwords are hashed before they are saved.

Fig. 3 shows the donor dashboard. The donor fills the form with the food name, type, number of servings, pickup address and deadline. After publishing, the donation appears in the donor's list and on the home page.

Fig. 4 shows an NGO account claiming a donation. After the claim, the status changes from Available to Claimed, and the donor's phone number is shown to the NGO for pickup.

Fig. 5 shows the final stage. When the donor clicks Mark delivered, the status changes to Delivered and the servings are added to the live counter on the home page. Donations that cross their pickup deadline without being claimed are shown as Expired and are removed from the public list.

Implementation Steps

1. **Problem Identification:** Surplus food is wasted while many people go hungry. The aim was to build a platform that connects donors with NGOs and needy people.

2. **Requirement Analysis:** Three user roles were identified (Donor, NGO, Needy) along with the actions allowed for each role.

3. **System Design:** The system was divided into a frontend (single-page interface), a backend (REST API) and a data store (JSON database). Users and donations are the two main data entities.

4. **Backend Development:** The Express server was created with routes for registration, login, donations, claiming, delivery, deletion and statistics.

5. **Authentication and Security:** Passwords were hashed using bcrypt. A JWT token is issued at login and verified on each protected route, and role checks stop users from performing actions that do not belong to their role.

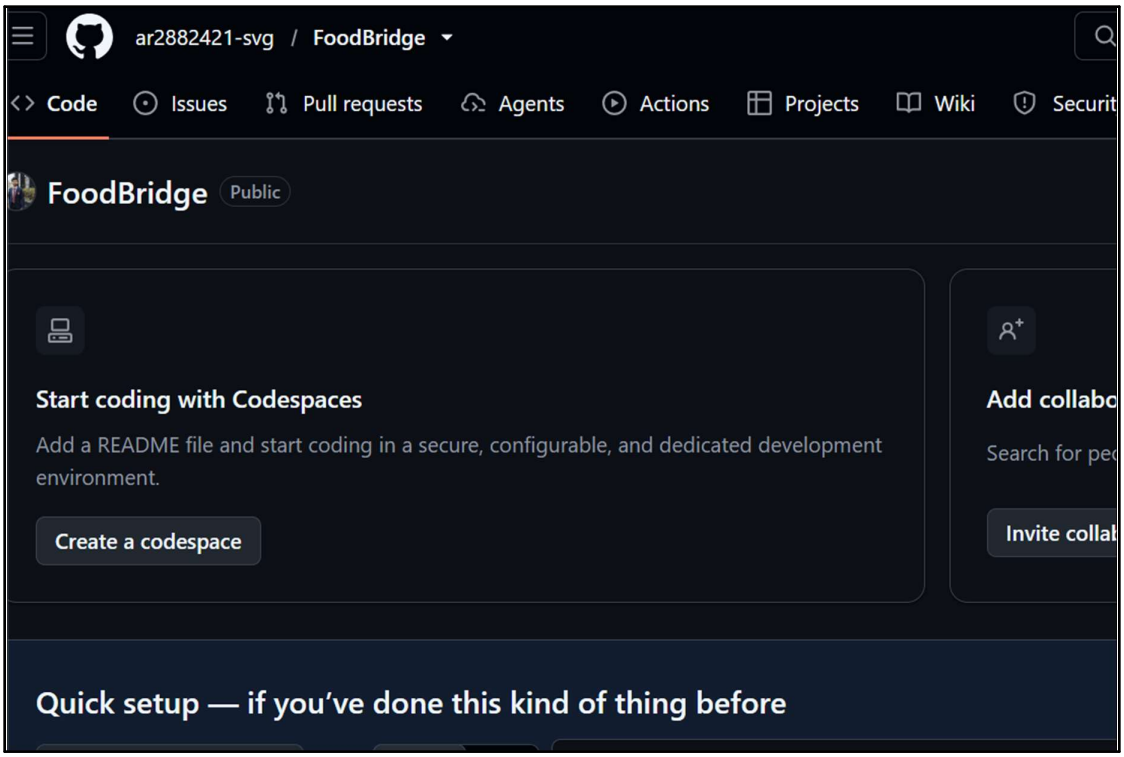
6. **Frontend Development:** The interface was built using HTML, CSS and JavaScript. The Fetch API connects it to the backend, and the page updates without reloading.

7. **Business Rules:** A donation can be claimed only once. Only available donations can be deleted. Only claimed donations can be marked delivered. Donations past their deadline are shown as expired automatically.

8. **Testing:** The system was tested by creating Donor, NGO and Needy accounts and running the complete flow from listing to delivery. Invalid inputs, wrong passwords, duplicate emails and unauthorised actions were also tested.

9. **Final Execution:** The application is started using npm install and npm start and runs at http://localhost:3000.

GitHub Screenshot



Link:- [https://github.com/ar2882421-svg/FoodBridge]

Conclusion

The FoodBridge project successfully demonstrates how a web application can help reduce food waste and support people in need. The platform lets donors list surplus food quickly and lets NGOs and needy people claim it before it expires.

The project combines a responsive frontend with a secure REST API backend. Important concepts such as client-server communication, password hashing, token-based authentication, role-based access control and input validation were implemented practically.

Overall, FoodBridge serves as a useful social-impact platform. In the future it can be enhanced by adding a production database such as MongoDB or MySQL, Google Maps for nearby donations, image upload for food photos, email and SMS notifications, a rating system for donors, an admin panel for verifying NGOs, and AI-based demand prediction.

References

[1] Node.js Foundation. (2025). Node.js Documentation. Available at: https://nodejs.org/en/docs

[2] Express.js. (2025). Express Documentation. Available at: https://expressjs.com/

[3] Mozilla Developer Network. (2025). Fetch API and JavaScript Guide. Available at: https://developer.mozilla.org/

[4] JWT. (2025). Introduction to JSON Web Tokens. Available at: https://jwt.io/introduction

[5] npm. (2025). bcryptjs Package. Available at: https://www.npmjs.com/package/bcryptjs

[6] W3Schools. (2025). HTML, CSS and JavaScript Tutorials. Available at: https://www.w3schools.com/

[7] GeeksforGeeks. (2025). Node.js and Express Tutorials. Available at: https://www.geeksforgeeks.org/